

Creational patterns

1. Singleton Pattern

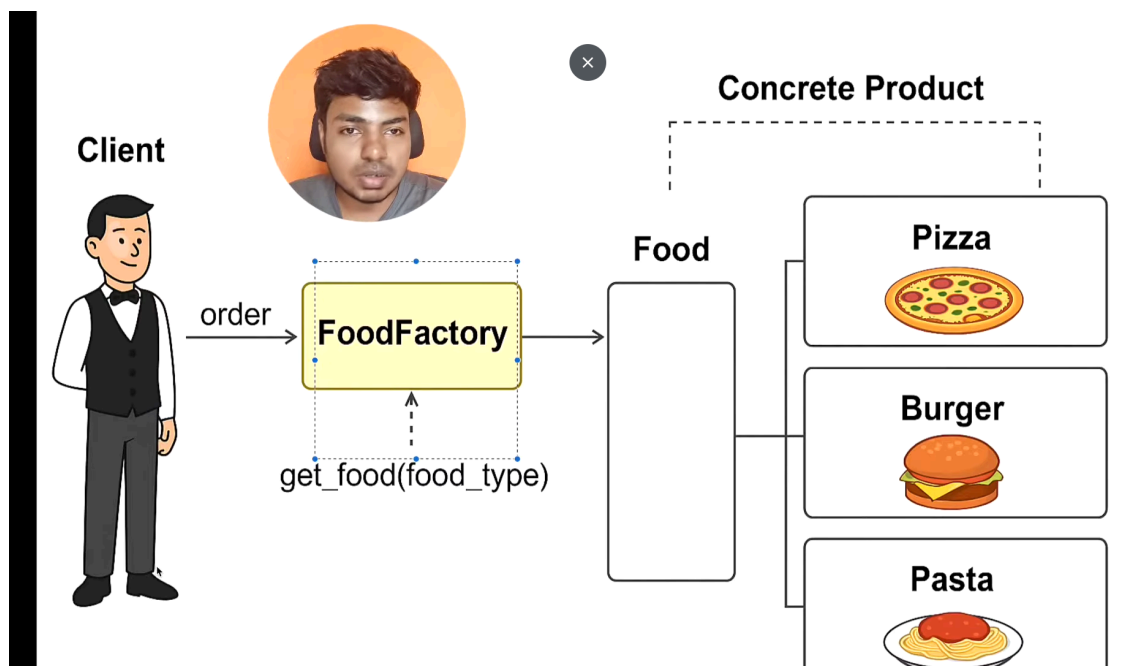
- Ensures that only one instance of a class is created and provides a global point of access to it.
- Useful when you want to have a single, shared instance of a class throughout the application.

2. Factory Method pattern

There will be multiple classes in an application, and we have to create objects for the classes.

Instead of giving the object creation logic directly to the clients, The interface will take care of creation of objects for the classes.

- It abstracts object creation process.
- Client only interacts with the common interface, not the concrete classes.



Abstracts object creation, but violates open/closed principle (open for extension, closed for modification). When we add another food, we have

to write if condition again for the food in the factory.

Instead we can have cheff for each food to prepare , just serve the food to customer by a servent.

3. Abstract Factory

Provides an interface to create families of objects without specifying their concrete classes.

Factory Method vs Abstract Factory

Factory Method

Creates **one type of product**.

Example:

```
VehicleFactory
├─ createVehicle()
│   ├── Car
│   └── Bike
```

Abstract Factory

Creates **multiple related products**.

Example:

```
GUIFactory
├─ createButton()
└─ createCheckbox()

WindowsFactory
├─ WindowsButton
└─ WindowsCheckbox

MacFactory
├─ MacButton
└─ MacCheckbox
```

One factory creates an entire family of UI components.

4. Builder Pattern

It allows to create complex objects step-by-step.

Assume there is a class with parameters. When we have to create an object we have to specify the value for the parameters to create the objects.

But every object there is not necessarily to have value for it, there are optional fields.

problems are:

1. remembering parameter order
2. many optional fields
3. constructor explosion (have to write constructor for accepting the value for parameter, in the order).

Problem (Before Builder)

Too many constructor parameters → hard to read and remember.

```
public class BankAccount {  
  
    private final String accountNumber;  
    private double balance;  
    private String branch;  
    private String accountType;  
  
    public BankAccount(  
        String accountNumber,  
        double balance,  
        String branch,  
        String accountType) {  
  
        this.accountNumber = accountNumber;  
        this.balance = balance;  
        this.branch = branch;  
        this.accountType = accountType;  
    }  
}
```

Usage:

```
BankAccount account =  
    new BankAccount(  
        "12345",  
        1000,  
        "Chennai",  
        "Savings");
```

✗ Hard to know what each parameter means.

Solution (After Builder)

Build object step by step.

```
public class BankAccount {  
  
    private final String accountNumber;  
    private double balance;  
    private String branch;  
    private String accountType;  
  
    private BankAccount(Builder builder) {  
        this.accountNumber = builder.accountNumber;  
        this.balance = builder.balance;  
        this.branch = builder.branch;  
        this.accountType = builder.accountType;  
    }  
  
    public static class Builder {  
  
        private String accountNumber;  
        private double balance;  
        private String branch;  
        private String accountType;  
  
        public Builder accountNumber(String accountNumber) {  
            this.accountNumber = accountNumber;  
            return this;  
        }  
  
        public Builder balance(double balance) {  
            this.balance = balance;  
            return this;  
        }  
  
        public Builder branch(String branch) {  
            this.branch = branch;  
        }  
    }  
}
```

```

returnthis;
    }

    publicBuilderaccountType(StringaccountType) {
        this.accountType=accountType;
        returnthis;
    }

    publicBankAccountbuild() {
        returnnewBankAccount(this);
    }
}

```

Usage:

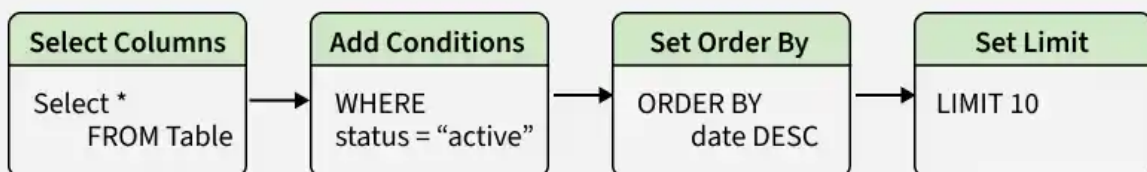
```

BankAccountaccount=
newBankAccount.Builder()
    .accountNumber("12345")
    .balance(1000)
    .branch("Chennai")
    .accountType("Savings")
    .build();

```

- ✓ Readable
- ✓ No constructor explosion
- ✓ Optional fields are easy

example:



Final Query : SELECT * FROM table WHERE status = "active" ORDER BY date DESC LIMIT 10

these are performed step-by-step